This project, **CanvasPDF AI**, will be your "Day 1" differentiator. Because your schedule is packed with CP and academics, we will treat this as a **"Module-Based Build."** Instead of trying to build the whole app at once, you will build one "Production-Ready" feature at a time.

By focusing on the **"Hard Parts"** (coordinates, file buffers, and state), you will gain the depth needed to ace technical interviews.

🛠️ Phase 1: The Robust Backend (Weeks 1-3)

Goal: Move beyond "vibe coding" by handling raw file data and secure authentication.

Week 1: Node.js Environment & File Streaming

- **Initialization:** Manually initialize your project with `npm init` and install `express`, `cors`, and `dotenv`.
- **Multer Implementation:** Use the `Multer` middleware to handle `multipart/form-data`. Do not just save files to a folder; learn how to handle **Memory Storage (Buffers)** versus **Disk Storage**.
- **PDF Extraction:** Use `pdf-parse` or `pdf-lib` to extract raw text and page counts from the uploaded buffer.
- **Interview Insight:** Be ready to explain the difference between a **Stream** and a **Buffer** when handling large PDF files.

Week 2: The "Brain" (Gemini 1.5 Pro Integration)

- **Prompt Engineering for Coordinates:** Structure your API call to Gemini 1.5 Pro. You aren't just asking for a summary; you are requesting a **JSON response** containing:

1. The explanation text.
2. The specific "Quote" from the PDF.
3. The **Normalized Bounding Box** $[ymin, xmin, ymax, xmax]$.

- **Context Window Management:** Leverage Gemini's 1M+ context window to upload the entire PDF once and hold a "Conversation" with the document.

Week 3: Database Schema & JWT Security

- **Mongoose Models:** Create a `User` schema and a `Document` schema. The `Document` schema should store the Cloudinary URL and an array of `Annotations` (coordinates + text).
- **JWT Flow:** Implement `jsonwebtoken` for login. Learn to use **HTTP-Only Cookies** for storing tokens—this is a high-security practice that interviewers love.

🎨 Phase 2: The Interactive Frontend (Weeks 4-6)

Goal: Mastering React state and the "Canvas" coordinate system.

Week 4: The Multi-Layered PDF Viewer

- **Rendering Layer:** Use `react-pdf` to render the document. You must handle "Virtualization" (only rendering pages in view) so the app doesn't crash on a 100-page PDF.
- **Interaction Layer:** Create a transparent `<div>` overlay (the Canvas) that sits exactly on top of the PDF page.
- **Coordinate Scaling:** Learn the math of **Viewport Scaling**. If the AI says a highlight is at `0.5, 0.5`, you must calculate how many pixels that is based on the current zoom level of the user's screen.

Week 5: Annotation UI & Tailwind Design

- **The "Pointing" System:** Build a React component that renders a "Textbox + Leader Line."
- **Dynamic Styling:** Use **Tailwind CSS** to make the annotations look modern. Add animations so that when an AI response arrives, the textbox "fades in" and points to the text.

Week 6: Global State (Context API)

- **State Management:** Use the **React Context API** to manage the `ActiveAnnotation`.
- **The "Jump" Feature:** Clicking an AI comment in the sidebar should trigger a function that calculates the page number and scrolls the `react-pdf` viewer to that specific location.

🚀 Phase 3: Agentic Features & Deployment (Weeks 7-8)

Goal: Final polish and "Day 1" resume bullets.

Week 7: Agentic Workflows

- **Function Calling:** Give the AI a "tool" to look up external definitions or Wikipedia links if the PDF doesn't explain a term well enough.
- **History Persistence:** Ensure the "Chat History" is saved in MongoDB so the user can return to their annotated PDF later.

Week 8: Production Readiness

- **Environment Variables:** Secure your API keys using `.env`.
- **Deployment:** Deploy the backend to **Render** and the frontend to **Vercel**.
- **Documentation:** Write a GitHub README that explains your **Coordinate Translation Logic**. This is what you will show during your internship interviews.

💡 How to fit this into your "Packed Schedule"

- **The 1-Hour Rule:** Spend 1 hour on this project *before* starting your CP streak. Use the AI to explain the logic of a library (like `Multer`), then write the implementation yourself.
- **Weekend Sprints:** Use Saturday mornings for the "Hard" logic (Coordinates/JWT) and Sunday nights for "Easy" tasks (Tailwind UI).